Extending Trustworthy Natural Language Interfaces for Quantum Optimization v3.3

by

Enoch Chan, Benjamin Moks, Kyle Ponte

Stevens.edu

December 3, 2025

Extending Trustworthy Natural Language Interfaces for Quantum Optimization v3.3

Enoch Chan, Benjamin Moks, Kyle Ponte
Stevens.edu

This document provides the requirements and design details of the PROJECT. The following table (Table 1) should be updated by authors whenever major changes are made to the architecture design or new components are added. Add updates to the top of the table. Most recent changes to the document should be seen first and the oldest last.

Table 1: Document Update History

Date	**Updates**
11/20/2025	K.P., E.C., B.M.: • Added the Development View chapter (Chapter 11), including the deployment diagram. • Added the Implementation View chapter (Chapter 10), containing the full class diagram. • Added the Logical View chapter (Chapter 8), including the package diagram for the system architecture. • Added the User Interface Design chapter (Chapter 7), which now contains personas and the first UI prototype frames.
11/14/2025	K.P., E.C., B.M.: • Updated the Weekly Reports chapter (Chapter A.11) to include week 11's report (A.1).
11/07/2025	K.P., E.C., B.M.: • Updated the Weekly Reports chapter (Chapter A.11) to include week 10's report (A.2). • Added project budget details and justification for API key costs. • Met with D-Wave Quantum advisor to discuss Voyager application and potential integration. • Configured a GitHub Actions autobuild pipeline that reads a Major.Minor.Changelist version (e.g., 1.0.1) from `version.txt`, renames the report PDF, and archives it into a versioned ZIP artifact. • Captured and added a screenshot of the successful autobuild run showing the versioned artifact (Figure A.1).

Table 1: Document Update History

Date	Updates
10/31/2025	K.P., E.C., B.M.: • Added system-wide Use Case Diagram in Chapter 6.3.5. • Created Activity Diagrams for each use case in Chapter 6.3.5. • Updated Non-Functional, System, and Domain Requirements in Chapter 5.1. • Linked all Use Cases, User Stories (Chapter 5), and Requirements (Chapter 5.1) with full traceability.
10/24/2025	K.P., E.C., B.M.: • Updated the Weekly Reports chapter (Chapter A.11) to include week 8's report (A.4). • Enlarged figures for better view. • Added key concepts to the glossary. • Incorporated dynamic numbering and labeling to use cases and user stories. • Added screenshots showcasing the directory of built packages.
10/17/2025	K.P., E.C., B.M.: • Updated the Weekly Reports chapter (Chapter A.11) to include week 7's report (A.5). • Linked use cases to requirements.
10/9/2025	E.C., B.M., K.P.: • Updated the Weekly Reports chapter (Chapter A.11) to include week 6's report. • Added chapters for the Requirements (Chapter 5.1), User Stories (Chapter 5), and Use Cases (Chapter 6.3.5). • Updated items and links to Glossary.
10/3/2025	K.P.: • Updated the Weekly Reports chapter (Chapter A.11) to include week 5's report. • Integrated a figure showcasing the pipeline's architecture.
09/25/2025	K.P.: • Updated the Weekly Reports chapter (Chapter A.11) to include week 4's report. • Integrated a section describing previous work and our plans within the Introduction chapter (Chapter 1).
09/19/2025	E.C., B.M., K.P.: • Updated the Weekly Reports chapter (Chapter A.11) to include week 3's report.
09/12/2025	E.C., B.M., K.P.: • Updated the Weekly Reports chapter (Chapter A.11) to include week 2's report.

Table 1: Document Update History

Date	Updates
09/04/2025	E.C., B.M., K.P.: • Added chapters on the introduction (Chapter 1) and weekly reports (Chapter A.11).

Table of Contents

List of Tables

List of Figures

Chapter 1

Introduction

– Chan, Moks, Ponte

1.1 About the Team

1.1.1 Enoch Chan

Enoch is pursuing a Bachelor of Engineering in Software Engineering. He is particularly fascinated by how intelligent systems can enhance automation, decision-making, and user experience in various domains. His interests extend to machine learning, algorithm optimization, and data-driven applications, where he enjoys exploring innovative ways to improve system efficiency. This project presents an exciting opportunity for Enoch to apply his knowledge, grow as a developer, and contribute meaningfully to the team's success.

1.1.2 Benjamin Moks

Ben is pursuing a Bachelor of Engineering in Software Engineering, with a keen interest in system architecture, scalable software design, and real-time computing. He enjoys tackling complex problem-solving challenges, optimizing software performance, and ensuring seamless integration between different technologies. This project provides an exciting opportunity for him to refine his technical skills, collaborate effectively, and contribute to building a robust and user-friendly solution.

1.1.3 Kyle Ponte

Kyle is currently pursuing a Bachelor of Engineering in Software Engineering, with a growing interest in data science and machine learning. He thrives on tackling complex challenges and uncovering innovative solutions, and he believes that collaboration and diverse perspectives drive meaningful progress. Eager to continually refine his skill set, Kyle sees this project as an opportunity to apply his knowledge, practice agile methodologies, and assist in the team's success.

Chapter 2

Team Declaration

– Chan, Moks, Ponte

Project Proposal

Our project focuses on developing a secure, user-friendly translation framework that enables non-experts to leverage the power of quantum optimization. Specifically, we aim to build upon recent research in natural language interfaces for quantum computing by designing a modular, object-oriented system that translates high-level problem descriptions into Quadratic Unconstrained Binary Optimization (QUBO) models suitable for execution on quantum solvers.

The proposed platform bridges the gap between everyday users and complex quantum technologies by providing an intuitive, natural language interface enhanced with validation, error detection, and semantic verification. By integrating object-oriented principles, we intend to improve system maintainability, scalability, and extensibility while ensuring that the translation process remains trustworthy and efficient. Our long-term vision is to create a framework that can be expanded beyond academic use to practical applications in logistics, scheduling, and resource allocation.

Mission Statement

The mission of our project is to lower the barrier to quantum computing adoption by creating a reliable natural language interface that translates optimization problems into QUBO formulations. This addresses the business need for organizations to explore quantum computing without requiring specialized expertise, while also supporting the business intent of enabling faster, more efficient decision-making processes in industries such as transportation, healthcare, and operations management.

Key Drivers

Several factors motivate the pursuit of this project:

1. **Accessibility to Quantum Computing:** Currently, most quantum programming requires advanced mathematical and technical skills, creating a steep learning curve. By designing an intu-

itive natural language interface, we aim to make quantum optimization accessible to researchers, students, and professionals outside of quantum physics or computer science.

2. **Growing Business Demand:** Organizations across industries face increasingly complex optimization challenges such as delivery routing, workforce scheduling, and network design. Quantum computing offers promising solutions, but adoption is limited by the lack of approachable tools. Our project addresses this need by providing a bridge between user intent and quantum execution.

3. **Verification and Trustworthiness:** Large language models (LLM), while powerful, are prone to errors such as hallucination or structural inconsistencies. To ensure real-world applicability, our system introduces safeguards like schema validation, reverse translation, and semantic scoring. These features directly enhance trustworthiness, which is critical for industries where correctness and reliability cannot be compromised.

4. **Educational Value:** Beyond immediate applications, our platform serves as a learning tool to help students and professionals understand the relationship between high-level problem descriptions and quantum-ready models. This educational aspect expands the reach and long-term sustainability of the system.

Key Constraints

Despite its potential, our project must operate within several constraints:

1. **Resource Limitations:** Access to true quantum hardware (such as D-Wave annealers) is limited and expensive. As a result, our system will initially rely on simulators or cloud-based quantum services. This constraint requires us to optimize for efficiency, ensuring that the translation pipeline works reliably even with limited compute resources.

2. **LLM Reliability:** Large language models can produce inconsistent or incorrect outputs depending on the prompt or problem complexity. To address this, our design must incorporate multiple validation layers and fallback mechanisms, but we recognize that absolute accuracy cannot be guaranteed. This constraint influences how we evaluate success and measure system robustness.

3. **Complexity of Optimization Problems:** Not all optimization problems translate neatly into QUBO formulations. Problems involving intricate relational constraints may cause higher failure rates. This constraint drives us to focus on representative problem sets while gradually expanding to more challenging categories.

4. **Time and Scope:** As an academic project with a defined timeline, we must balance ambition with feasibility. While future work could include fine-tuning models or integrating with symbolic verification tools, our current milestone limits us to replicating and incrementally improving the existing framework.

Together, these constraints frame our project as both ambitious and pragmatic. By acknowledging them early, we position our team to deliver a functional, professional-grade prototype that demonstrates both technical feasibility and meaningful improvement upon prior work.

2.1 Background and Prior Work

Last year's group introduced the idea of a trustworthy natural language interface for quantum optimization, focusing on translating user prompts into structured JSON representations. Their system leveraged large language models to generate decision variables, constraints, and objectives, and they evaluated these outputs with schema validation to determine structural correctness. They tested across five canonical problem types—Knapsack, Traveling Salesman Problem (TSP), Graph Coloring, Seating Assignment, and Team Assignment—measuring success rates, failure categories, and latency. This work demonstrated that GPT-4 outperformed GPT-3.5 on several problems, achieving higher rates of schema-valid JSON and maintaining interactive performance times.

However, the prior group's implementation never extended beyond JSON validation. They did not complete the reverse translation step that compares the generated QUBO representation back to the original prompt for semantic fidelity. They also did not translate the validated JSON into QUBO form or run it on a quantum solver, leaving critical components of the proposed framework unimplemented. As a result, their evaluation was limited to whether JSON outputs were well-formed, without testing true semantic alignment or solver compatibility.

Our project builds directly on this foundation by addressing these gaps. Specifically, we are working to (i) implement reverse translation and fidelity scoring so that semantic accuracy can be quantitatively measured, (ii) translate valid JSON outputs into executable QUBO formulations, and (iii) extend the system with modular, object-oriented components for maintainability and scalability. By completing these steps, our goal is to move beyond structural validation and establish a pipeline that not only produces valid JSON but also ensures semantic correctness and quantum-executable outputs.

2.2 System Architecture

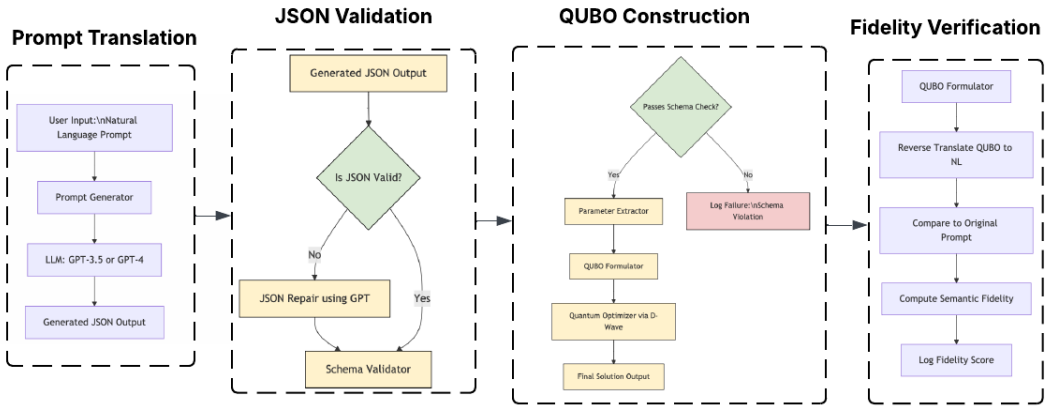

Figure 2.1: LLM-to-QUBO Pipeline Architecture

Chapter 3

Stakeholders

– Chan, Moks, Ponte

This chapter identifies the groups and individuals who will be directly or indirectly impacted by the system. Each stakeholder plays a specific role in ensuring the success, adoption, and continued improvement of the trustworthy natural language interface for quantum optimization.

3.1 Primary Stakeholders

End Users — Individuals who will use the interface to describe optimization problems in natural language and obtain quantum or classical solutions. They include students, researchers, and professionals in fields such as logistics, scheduling, and resource allocation.

Developers and Engineers — Team members and future contributors responsible for implementing and maintaining the translation pipeline, integrating solvers, and ensuring the reliability of the system architecture.

Academic Advisors and Sponsors — Faculty mentors and technical representatives from organizations such as D-Wave who provide research guidance, solver access, and ensure alignment with trustworthy-AI principles.

3.2 Secondary Stakeholders

Regulatory and Ethical Review Bodies — Institutional Review Boards (IRB) or equivalent entities that ensure compliance with data privacy and research ethics, particularly that no personal or sensitive data are used in prompts.

Third-Party API Providers — Organizations such as OpenAI and D-Wave that supply the APIs, SDKs, or solvers upon which the system depends.

Competitors and Industry Observers — Developers of comparable frameworks (e.g., OR-LLM-Agent, LLMOPT) who may use this system's outcomes as benchmarks for trustworthiness and usability in future research.

3.3 Stakeholder Impact Summary

The system impacts stakeholders by:

- Empowering end users to interact with quantum optimization through intuitive natural language input.
- Enabling developers to refine modular and verifiable architectures for hybrid quantum-classical systems.
- Providing academic sponsors and advisors with measurable research outputs for evaluation and publication.
- Supporting ethical compliance and responsible AI development through adherence to data privacy standards.
- Encouraging cross-institutional collaboration by releasing prompt libraries and experimental datasets.

Together, these stakeholders form a collaborative ecosystem that drives innovation, ensures accountability, and extends the practical reach of quantum optimization research.

Chapter 4

Requirements

– Chan, Moks, Ponte

4.1 Stakeholders

4.1.1 Customers

End users who want to describe optimization problems in plain English and obtain valid solutions through quantum/classical solvers.

4.1.2 Sponsors

Technical advisors from D-Wave supporting research in trustworthy AI/quantum optimization.

4.1.3 Engineering and Technical Persons

Developers implementing LLM interfaces, QUBO compilers, and solver integrations.

4.1.4 Regulators

IRB and institutional guidelines ensuring no sensitive personal data is used in prompts.

4.1.5 Third Parties

Providers of APIs (e.g., OpenAI, D-Wave) that supply underlying LLM or solver services.

4.1.6 Competitors

Other optimization frameworks (e.g., OR-LLM-Agent, LLMOPT) that provide similar functionality for classical optimization.

4.2 Key Concepts

QUBO: Quadratic Unconstrained Binary Optimization formulation used for optimization problems.

BQM: Binary Quadratic Model, solver-ready form of QUBO.

LLM: Large Language Model, used for translating natural language into QUBO JSON.

fidelity: A measure of how well reverse-translated text preserves the original intent.

Prompt Library: A curated set of optimization prompts used for benchmarking and testing.

4.3 User Requirements

Table 4.1: User Requirements Table

Requirement	Priority	Use Case(s)
Functional Requirement 1 (reqfTranslation) *The system shall accept a natural language prompt and generate schema-compliant JSON (QUBO representation).*	MustHave	UC_1
Functional Requirement 2 (reqfValidation) *The system shall validate JSON and classify errors such as missing objectives, malformed constraints, or hallucinated variables.*	MustHave	UC_1
Functional Requirement 3 (reqfReverseTranslate) *The system shall reverse translate a QUBO into natural language and compute a fidelity score against the original prompt.*	MustHave	UC_2
Functional Requirement 4 (reqfCompilation) *The system shall compile schema-compliant JSON into a QUBO and support five canonical optimization problem types.*	MustHave	UC_1, UC_3
Functional Requirement 5 (reqfExecution) *The system shall execute the compiled QUBO on a simulator (and optionally on a hardware solver) and return feasible solutions.*	MustHave	UC_3
Functional Requirement 6 (reqfReporting) *The system shall log prompts, outputs, metrics, and export results as CSV and plots.*	MustHave	UC_3, UC_4
Functional Requirement 7 (reqfPromptLibrary) *The system shall provide a prompt library where users can add, update, and tag benchmark prompts by problem type.*	ShouldHave	UC_5

4.4 Non-functional (Quality) Requirements

Table 4.2: Non-Functional (Quality) Requirements Table

Requirement	Priority	Use Case(s)
Quality Requirement 1 (reqqLatency) *The system shall achieve a median latency ≤ 6 seconds, with 95th percentile latency ≤ 9 seconds.*	MustHave	UC_4
Quality Requirement 2 (reqqFidelity) *The system shall achieve a fidelity score of at least 0.70 for automatic acceptance; lower fidelity shall be flagged.*	MustHave	UC_2
Quality Requirement 3 (reqqSchemaValid) *The system shall produce at least 80% schema-valid JSON across the five canonical problem types.*	MustHave	UC_1, UC_4
Quality Requirement 4 (reqqMaintainability) *The system shall maintain ≥ 85% unit test coverage, use typing, and follow modular object-oriented design practices.*	MustHave	UC_1, UC_2, UC_3, UC_4, UC_5
Quality Requirement 5 (reqqSecurity) *The system shall store no PII, keep API keys in secure secrets, and ensure all logs are sanitized.*	MustHave	UC_1, UC_2, UC_3, UC_4

4.5 System (Constraints) Requirements

Table 4.3: System (Constraints) Requirements Table

Requirement	Priority	Use Case(s)
Constraint Requirement 1 (reqcTechStack) *The system shall be implemented in Python 3.11 and use OpenAI SDK, pandas, jsonschema, matplotlib, and sentence-transformers.*	MustHave	UC_1, UC_2, UC_3
Constraint Requirement 2 (reqcBudget) *The system shall operate under budgeted API usage; default simulator shall be used, with hardware solvers optional.*	MustHave	UC_3
Constraint Requirement 3 (reqcData) *The system shall only use synthetic or public prompts (no IRB approval required).*	MustHave	UC_1, UC_2, UC_5

Table 4.3: System (Constraints) Requirements Table

Requirement	Priority	Use Case(s)
Constraint Requirement 4 (reqcLocalExecution) *The system shall be fully executable on the user's machine without requiring any web interface or network connectivity. At minimum, all core functionalities must run through a local terminal or command-line interface using the project source code.*	MustHave	UC_1, UC_3, UC_5
Constraint Requirement 5 (reqcLocalWebUI) *The system should provide an optional locally hosted web-based interface (e.g., running on localhost) to allow users to interact with the system via a graphical UI. This interface should not depend on any external deployment and must run entirely on the user's machine.*	ShouldHave	UC_1, UC_3, UC_5

4.6 Domain (Business) Requirements

Table 4.4: Domain (Business) Requirements Table

Requirement	Priority	Use Case(s)
Business Requirement 1 (reqbResearch) *The system shall be designed for academic and research purposes only and not for commercial deployment.*	MustHave	UC_1, UC_2, UC_3, UC_4, UC_5

Chapter 5

User Stories

– Chan, Moks, Ponte

5.1 User Stories

Each user story is linked to its associated Use Case(s) and Requirement(s) to ensure traceability throughout the system design.

Table 5.1: User Stories (linked to Use Cases and Requirements)

ID	Story (Role • Goal • Benefit)	Use Case(s)	Requirement(s)
US-1	As an operations analyst with no quantum background, I want to type a routing/scheduling problem in plain English and immediately receive a solver-ready model so I can compare alternatives without learning QUBO syntax. • **Acceptance**: Given a valid NL prompt, the system returns schema-valid JSON and a compiled QUBO. • Validation errors are reported with reason.	UC_1, UC_3	$reqkFunctional_1$, $reqkFunctional_4$
US-2	As a stakeholder, I need the system to show how my request was interpreted and a fidelity score so I can trust the optimization reflects my intent. • **Acceptance**: Reverse-translated text is shown side-by-side with the original; fidelity score reported. • Scores below threshold are flagged.	UC_2, UC_4	$reqkFunctional_3$, $reqkQuality_2$
US-3	As a planner, I want the system to execute on a simulator and report the best-found solution with constraint checks and runtime so I can decide. • **Acceptance**: Solver returns objective value, assignment, feasibility status, and runtime. • Results are saved for reuse.	UC_3	$reqkFunctional_5$, $reqkFunctional_6$

Table 5.1: User Stories (linked to Use Cases and Requirements)

ID	Story (Role • Goal • Benefit)	Use Case(s)	Requirement(s)
US-4	As a researcher, I need logs, metrics, and versioned prompts so results are reproducible across runs and models. • **Acceptance**: CSV/plots are exported; runs recorded with model version, seeds, and timestamps. • Prompt versions are tracked and retrievable.	UC_4, UC_5	$reqkFunctional_6$, $reqkFunctional_7$, $reqkQuality_4$

Chapter 6

Use Cases

Chan, Moks, Ponte

This chapter presents the use case diagrams that satisfy the system requirements. Each use case is mapped to at least one requirement, with preconditions, flows, and postconditions defined. All use cases assume the system is run locally, either through a command-line interface, Python environment, or an optional locally hosted interface on `localhost`.

6.1 Table of Use Cases

Table 6.1: Use Cases Table

Use Case	Requirements	Name and Description	Activity Diagrams
UC_1	$reqkFunctional_1$, $reqkQuality_3$	ucTranslatePrompt: Convert a natural language optimization prompt into schema-valid JSON and compile it into a QUBO model ready for a solver.	6.2
UC_2	$reqkFunctional_3$, $reqkQuality_2$	ucReverseTranslate: Reconstruct natural language from the QUBO/JSON and compute a fidelity score to show how well it matches the user's intent.	6.3
UC_3	$reqkFunctional_5$, $reqkFunctional_6$	ucExecuteReport: Run the QUBO on a simulator or solver and return the best solution, constraint satisfaction, and runtime, saving outputs for reuse.	6.4
UC_4	$reqkFunctional_6$, $reqkQuality_1$, $reqkQuality_3$	ucViewLogsMetrics: Display and download logs, success rates, latency, and fidelity plots to support reproducibility and analysis.	6.5
UC_5	$reqkFunctional_7$	ucManagePromptLibrary: Allow the team to add, update, and tag benchmark prompts by problem type for consistent testing.	6.6

6.2 Detailed Use Case Examples

Table 6.2: . ucTranslatePrompt (UC_1)UC-1 Translate NL to QUBO

Use Case 1 (ucTranslatePrompt (UC_1)UC-1 Translate NL to QUBO) *UC-1 Turns a natural language optimization prompt into schema-valid JSON and compiles it into a QUBO model.*
Requirements: $reqkFunctional_1$, $reqkQuality_3$
Brief description: User provides a plain-English prompt (e.g., "Assign 20 technicians to 4 shifts") and the system generates schema-valid QUBO JSON plus a compiled QUBO.
Primary actors: User
Secondary actors: LLMClient, QUBOTranslator, QUBOCompiler
Preconditions: User has launched the local system interface (CLI, Python script, or locally hosted UI); LLM service available.
Main flow: 1. User submits natural language prompt. 2. System translates to JSON. 3. JSON validated against schema. 4. Compiled into QUBO model.
Postconditions: Valid QUBO model ready for solving.
Alternative flows: Invalid JSON → error message; compilation fails → fallback.

Table 6.3: . ucReverseTranslate (UC_2)UC-2 Reverse Translate & Score

Use Case 2 (ucReverseTranslate (UC_2)UC-2 Reverse Translate & Score) *UC-2 Reconstruct a QUBO into natural language and compute a fidelity score.*
Requirements: $reqkFunctional_3$, $reqkQuality_2$
Primary actors: User
Secondary actors: LLMClient, QUBOReverseTranslator, FidelityCalculator
Preconditions: Valid QUBO JSON available.
Main flow: 1. System takes compiled QUBO JSON. 2. Reverse translator reconstructs natural language description. 3. FidelityCalculator computes similarity against the original prompt.
Postconditions: Fidelity score available to the user.
Alternative flows: Low fidelity score → flagged result with warning.

Table 6.4: . ucExecuteReport (UC_3)UC-3 Execute & Report

Use Case 3 (ucExecuteReport (UC_3)UC-3 Execute & Report) *UC-3 Run QUBO on simulator/solver and return solutions with reports.*
Requirements: $reqkFunctional_5$, $reqkFunctional_6$
Primary actors: User
Secondary actors: LocalSolver, DWaveSolver, ResultWriter
Preconditions: Compiled QUBO available.
Main flow: 1. User requests execution of compiled QUBO. 2. System runs QUBO on simulator by default. 3. (Optional) If hardware solver is available, extend to D-Wave execution. 4. Collect objective value, assignment, feasibility, and runtime. 5. Save results and metrics for later retrieval.
Postconditions: Results stored and available for analysis.
Alternative flows: Solver unavailable → error returned; execution timeout → fallback.

Table 6.5: . ucViewLogsMetrics (UC_4)UC-4 View Logs & Metrics

Use Case 4 (ucViewLogsMetrics (UC_4)UC-4 View Logs & Metrics) *UC-4 View or download logs, success rates, latency, and fidelity plots.*
Requirements: $reqkFunctional_6$, $reqkQuality_1$, $reqkQuality_3$
Primary actors: User
Secondary actors: ResultMetrics, Visualization
Preconditions: Logs and metrics exist from previous executions.
Main flow: 1. User requests logs and metrics. 2. System fetches stored data. 3. System displays results through the local interface (CLI output, local visualization window, or locally hosted dashboard) and/or allows export as CSV/plots.
Postconditions: Logs and visualizations delivered to user.
Alternative flows: Logs unavailable → system returns "no data available."

Table 6.6: . ucManagePromptLibrary (UC_5)UC-5 Manage Prompt Library

Use Case 5 (ucManagePromptLibrary (UC_5)UC-5 Manage Prompt Library) *UC-5 Add, update, and tag benchmark prompts by problem type.*
Requirements: $reqkFunctional_7$
Primary actors: User
Secondary actors: PromptDataset
Preconditions: Prompt library available.
Main flow: 1. User adds a new prompt or selects an existing one. 2. User edits or tags the prompt with problem type. 3. System saves changes and updates library.
Postconditions: Prompt library updated successfully.
Alternative flows: Invalid prompt → rejected with error; duplicate prompt → flagged for review.

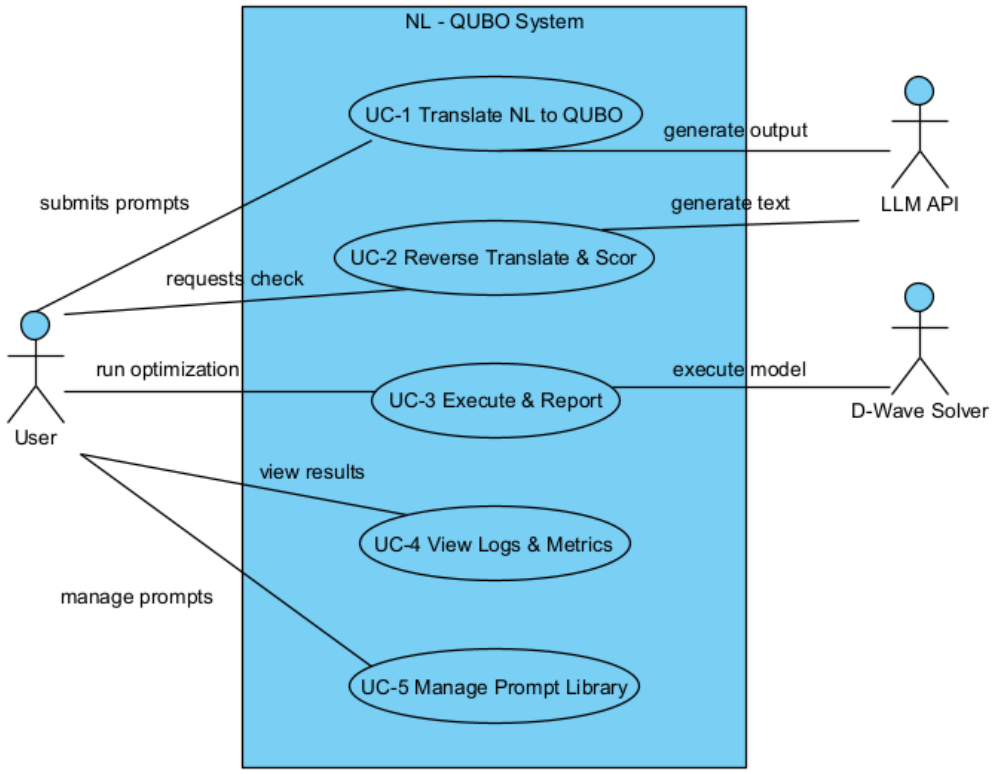

Figure 6.1: Use Case Diagram for the NL–QUBO System showing the five core user interactions.

6.3 Activity Diagrams

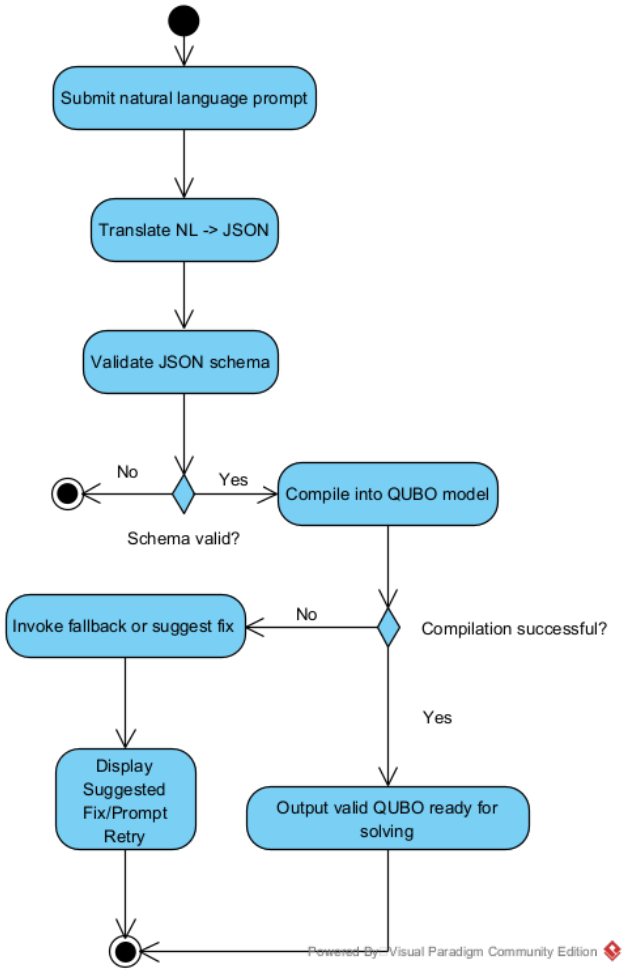

Figure 6.2: Activity Diagram for ucTranslatePrompt (UC_1)

6.3.1 Activity Diagram 1 Description

In this activity, the workflow begins when the user interfaces with the system through the main controller or UI, which passes the written natural-language prompt into the system as a Prompt object. The LLMClient sends this prompt to the language model, while the QUBOTranslator structures the request, parses the response, and converts the returned text into a JSON-formatted representation of variables, constraints, and the optimization objective. Once JSON is obtained, a validation step is performed by SchemaValidator to ensure that the output conforms to the expected QUBO schema. After validation succeeds, the QUBOCompiler converts the JSON into an internal QUBOModel (or BQM) that stores coefficients and variable names, making it executable by downstream solver classes.

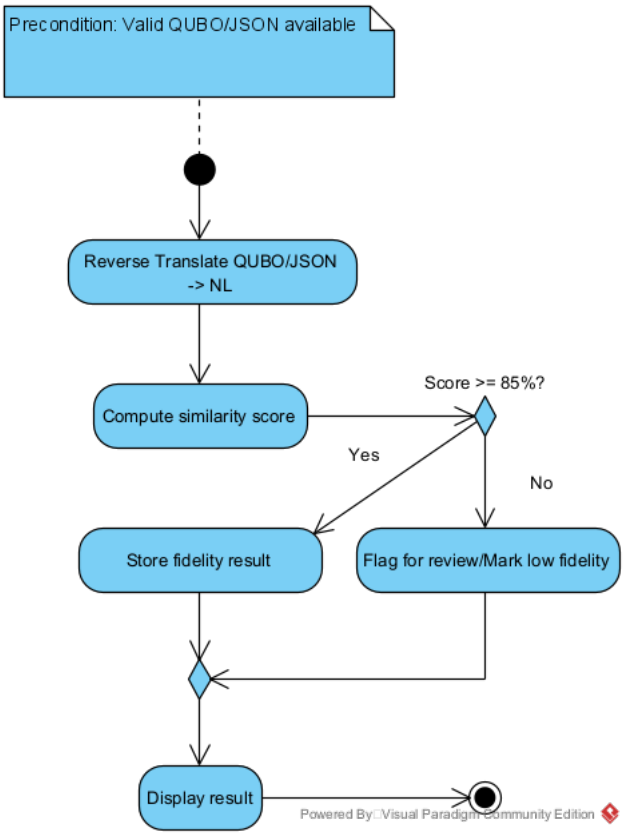

Figure 6.3: Activity Diagram for ucReverseTranslate (UC_2)

6.3.2 Activity Diagram 2 Description

This activity begins with the previously compiled QUBOModel, selected by the UI or controller. The QUBOReverseTranslator inspects the model's internal structure and constructs an LLM prompt that describes the model's objective and constraints, relying again on the LLMClient to convert the structured representation back into natural language. The output description is then passed to the FidelityCalculator, which compares it against the original Prompt and computes a similarity score based on embeddings, token overlap, or other comparison metrics. The resulting fidelity score, along with the reconstructed description, is stored in Result and ResultMetrics objects so the system can track model interpretability and correctness across runs.

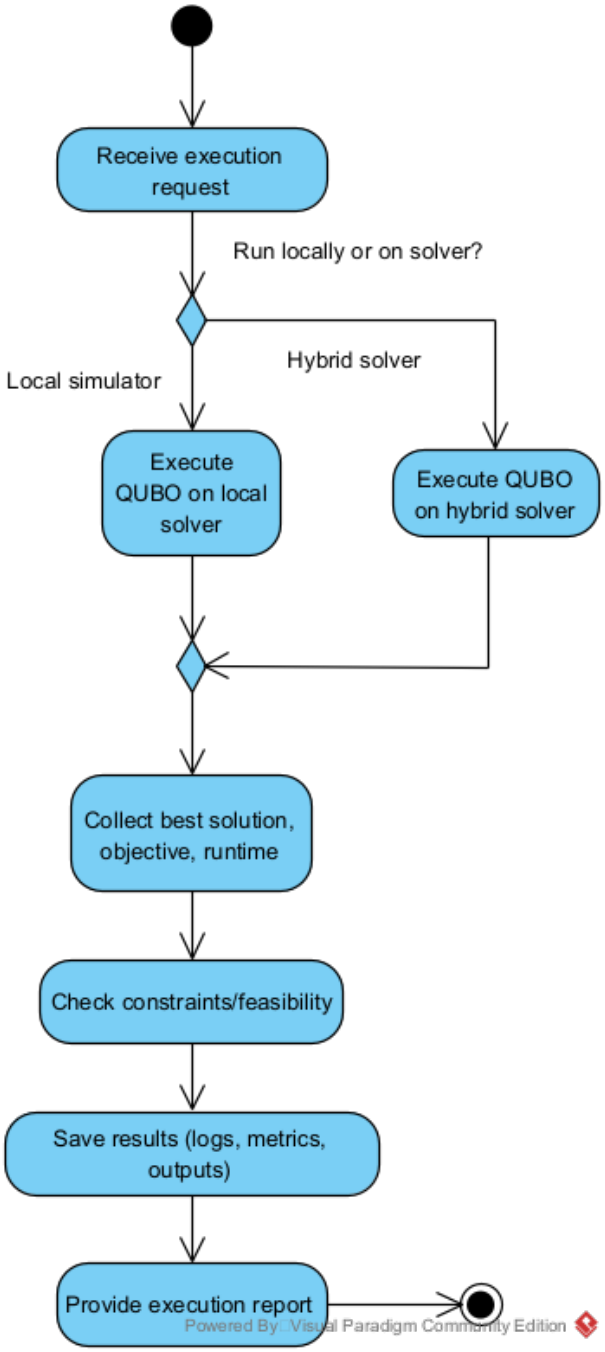

Figure 6.4: Activity Diagram for ucExecuteReport (UC_3)

6.3.3 Activity Diagram 3 Description

When the user chooses to execute a compiled model, the UI/controller retrieves the corresponding QUBOModel and dispatches it to the chosen solver. The default solver is the LocalSolver, which performs classical simulation on the model, whereas DWaveSolver may be invoked for hardware execution if configured. These solver classes evaluate the model using the stored coefficients and return solutions including

binary assignments, objective values, constraint violations, timing information, and hardware metadata. The controller then passes solver output to Result and ResultMetrics, which store structured data about performance and correctness. Finally, ResultWriter persists these results to files or logs so they can be aggregated, visualized, and compared across experiments.

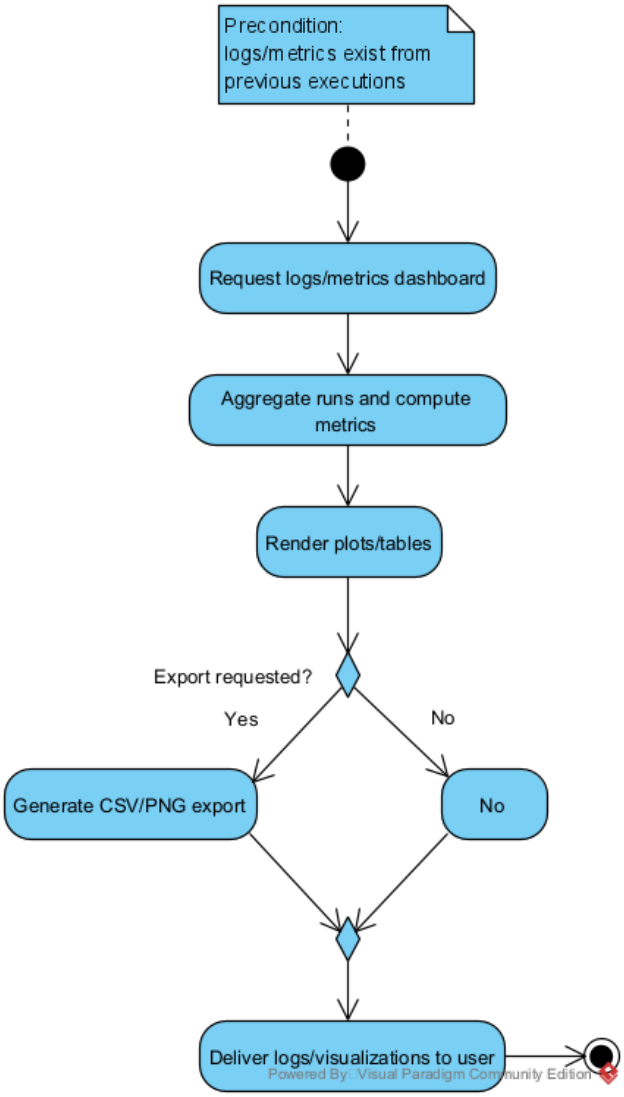

Figure 6.5: Activity Diagram for ucViewLogsMetrics (UC_4)

6.3.4 Activity Diagram 4 Description

Here, the user interacts through the UI/controller to request previous results or analytics. The controller queries ResultMetrics to retrieve aggregated statistics such as fidelity distributions, runtime comparisons, success rates, and solver-to-solver differences. Raw logs and run histories come from stored Result objects and files written by ResultWriter. When the user requests visual output, the Visualization class generates charts (e.g., histograms, scatter plots, line graphs) or exports tables suitable for CSV/analysis pipelines. The UI then displays these results in textual or graphical form, allowing users to compare runs or export data.

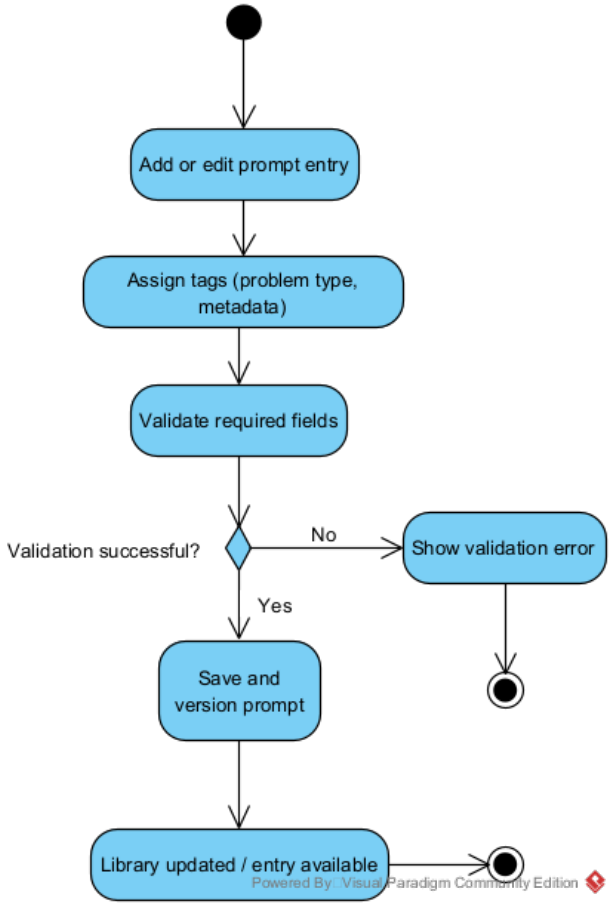

Figure 6.6: Activity Diagram for ucManagePromptLibrary (UC_5)

6.3.5 Activity Diagram 5 Description

This activity allows users to create, edit, tag, or select existing prompts for benchmarking. The UI/-controller enables selection from the PromptLibrary, which store all prompt templates along with metadata like problem type, tags, and optional ground-truth information. When editing occurs, the Prompt object is updated with new structure or content, and changes are stored back into the dataset. If prompts are linked to experiment runs or fidelity evaluations, ResultWriter may log updates so comparisons between model versions remain traceable. The PromptLibrary ensures that prompts remain organized and retrievable across future workflows like translation, execution, and reverse translation.

Chapter 7

User Interface Design

– Chan, Moks, Ponte

7.1 User Persona

7.1.1 Persona 1: Alex Ramirez

Role: Logistics Planner

Goal: Solve real-world scheduling and routing problems without needing to learn quantum computing.

Needs: Simple natural-language input that produces trustworthy QUBO output.

Tasks:

- Enter problem in plain English.
- Review the generated JSON and validation messages.
- Run the solver and view results.
- Export solutions.

Scenario:
Alex types: *"Assign 20 drivers to 4 routes with balanced workload."* The system generates JSON, compiles a QUBO, shows the fidelity score, executes the solver, and outputs a balanced assignment.

7.1.2 Persona 2: Dr. Maya Lee

Role: Operations Researcher

Goal: Run experiments and gather reproducible performance metrics.

Needs: Prompt library, logs, CSVs, and latency/fidelity plots.

Tasks:

- Add or edit benchmark prompts.
- Run multiple tests.
- Review logs and computed metrics.
- Export CSV files and visual plots.

Scenario:
Dr. Lee tests several Traveling Salesman and Graph Coloring prompts to compare how different LLMs behave. She collects fidelity scores, success rates, and runtime plots. She uses the exported CSVs and graphs as evidence in her research paper on trustworthy quantum optimization interfaces.

7.2 User Interface Prototype

This section presents the **first prototype** of the user interface for Team Extreme's NL–QUBO Translation & Execution System. The goal of this prototype is to show the end-to-end flow from entering a natural language prompt, through translation and fidelity checking, to executing the compiled QUBO and viewing the solver results. The current design is a working draft and will be refined based on feedback and usability testing.

7.2.1 Frame 1: Prompt Translation and QUBO Summary

Figure 7.1 shows the first screen of the prototype. The user types a natural language request (for example, "Assign 20 technicians to 4 shifts so that each shift has at least 4 people.") into the prompt box at the top. The system then walks the user through the pipeline: translating the prompt with the LLM, validating the JSON, and compiling it into a QUBO. A short QUBO summary (variables, objective, constraints, and model size) is displayed so the user can see what the system built. The interface then performs reverse translation and computes a fidelity score, so the user can judge whether the QUBO matches their original intent before moving on to execution.

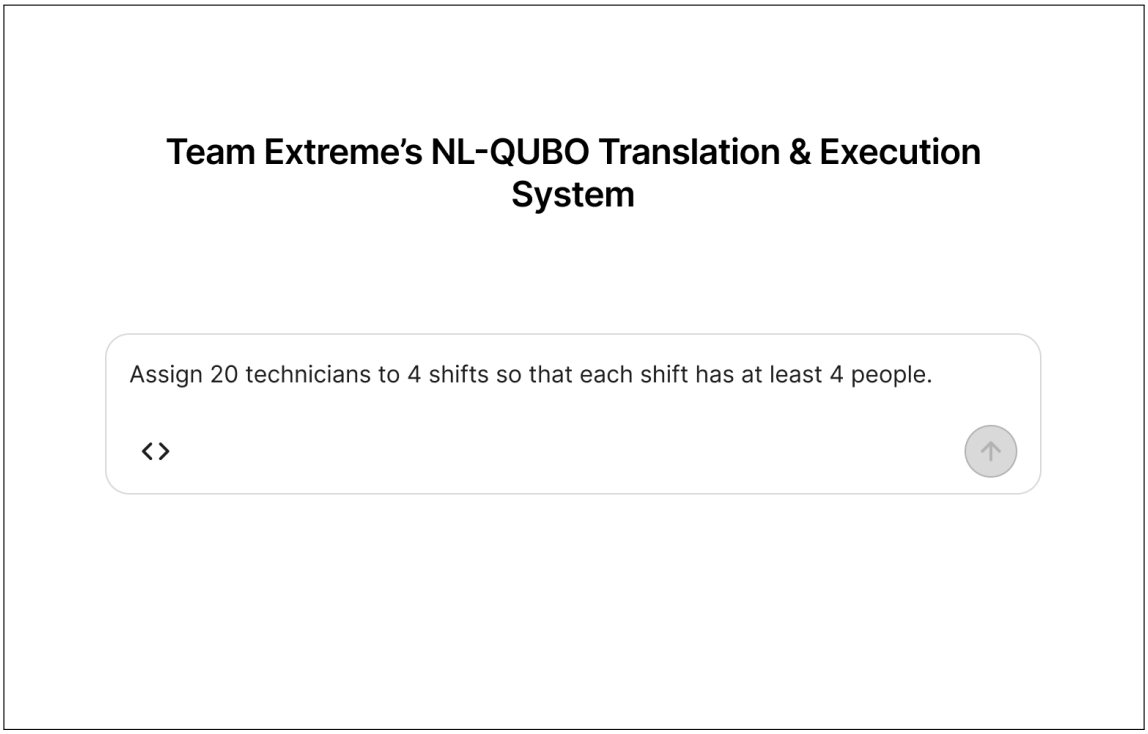

Figure 7.1: First UI prototype: prompt entry, translation status, QUBO summary, and fidelity score.

7.2.2 Frame 2: Local Solver Execution and Results

Figure 7.2 shows the second screen of the prototype, which appears after the user chooses to proceed to execution. The system runs the compiled QUBO on a local solver and reports the execution status, the best objective value (for example, overtime hours), feasibility, runtime, and a short solution preview that shows how technicians are assigned to each shift. At the bottom, the UI asks whether the user would like to save logs and metrics, preparing the transition to the logging and analysis features.

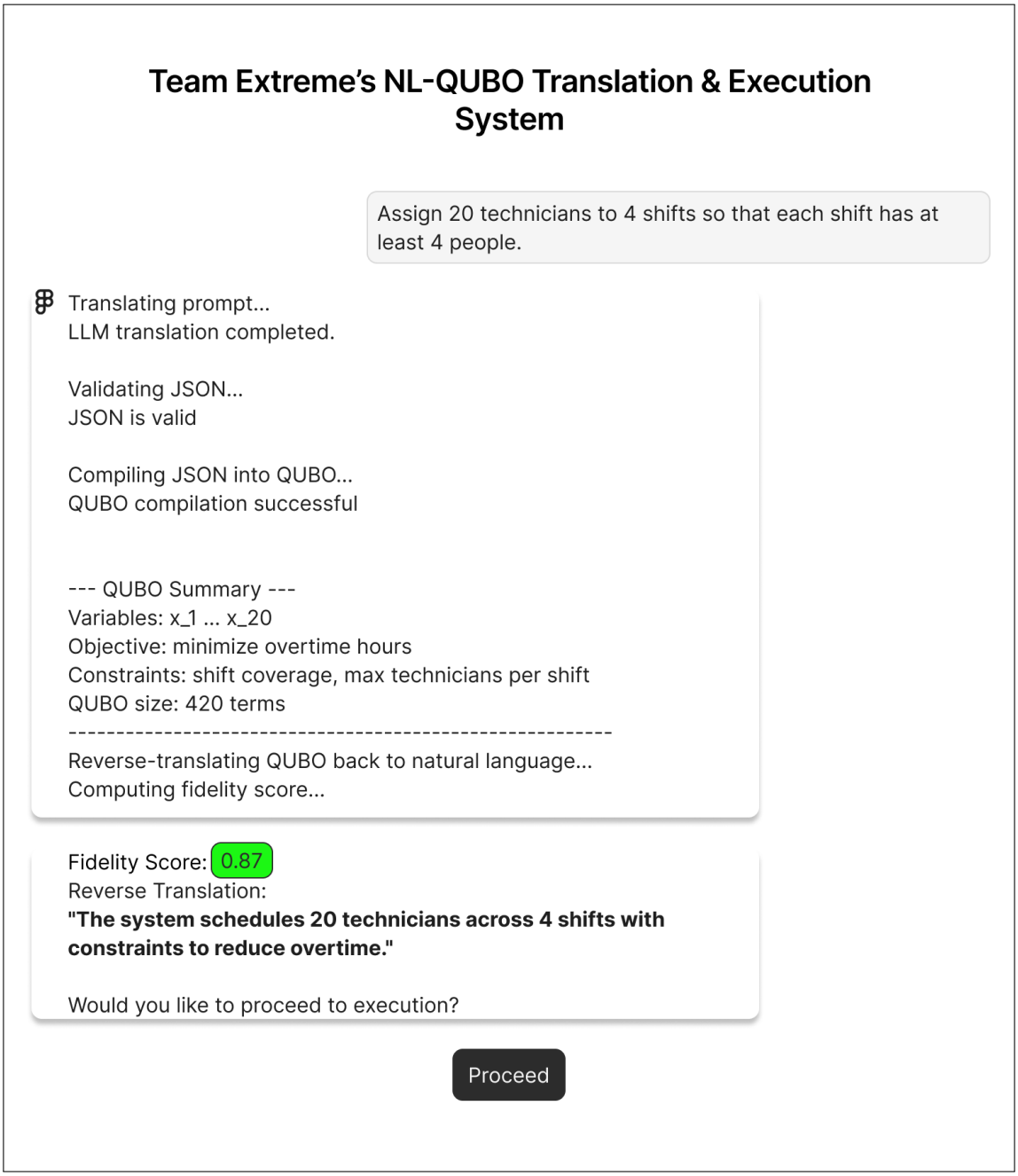

Figure 7.2: First UI prototype: local solver execution results and solution preview.

7.2.3 Frame 3: Logs and Metrics (Prototype Placeholder)

Figure 7.3 represents the third frame of the first UI prototype, focused on viewing and exporting logs and metrics from previous runs. In the final system, this screen will allow users to review success rates, latency, fidelity scores, and other performance metrics across multiple experiments, as well as export CSV files or plots for further analysis. In this initial prototype, the layout is mainly illustrative and will be refined once the logging back end and metrics pipeline are fully implemented.

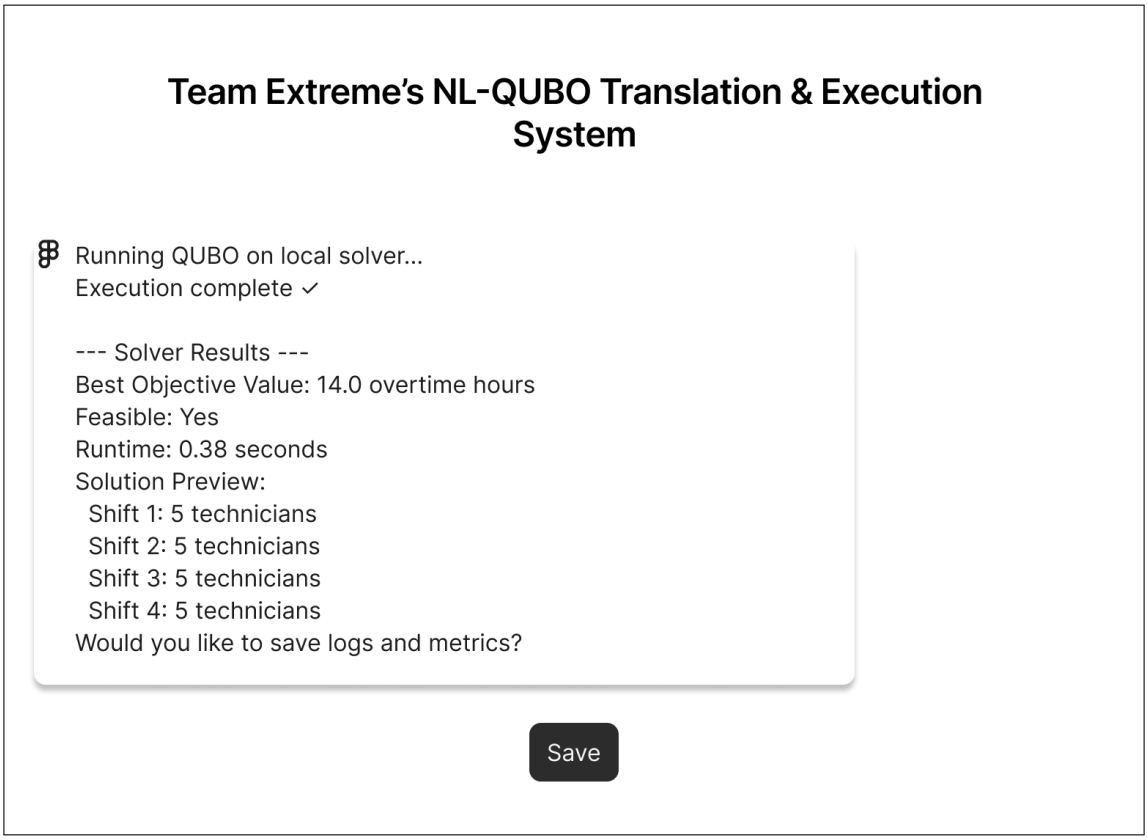

Figure 7.3: First UI prototype: conceptual layout for logs and metrics visualization and export.

Chapter 8

Logical View

– Chan, Moks, Ponte

8.1 Package Diagram

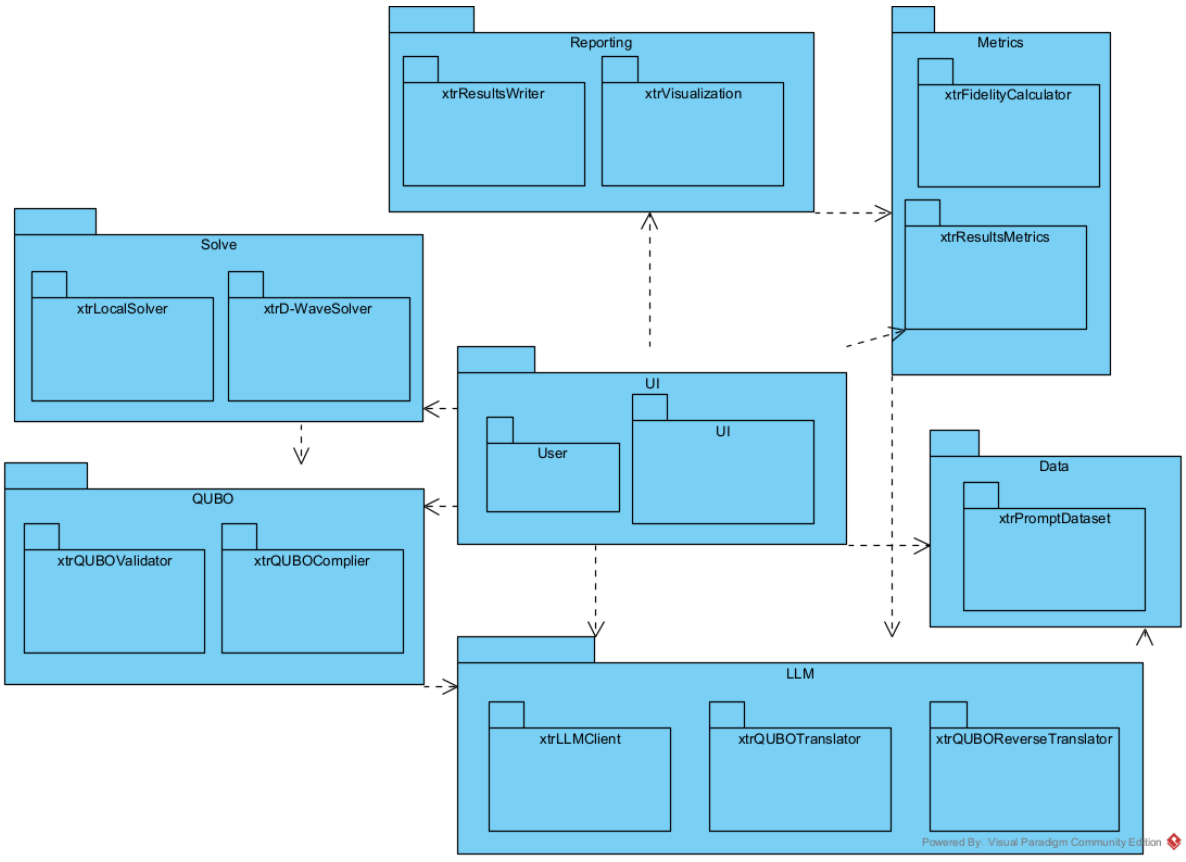

Figure 8.1: Package Diagram

Chapter 9

Process View

– Chan, Moks, Ponte

Choose a proper UML diagram, such as activity diagram, sequence diagram, etc. to show the dynamic aspects of your system.

9.1 Sequence Diagrams

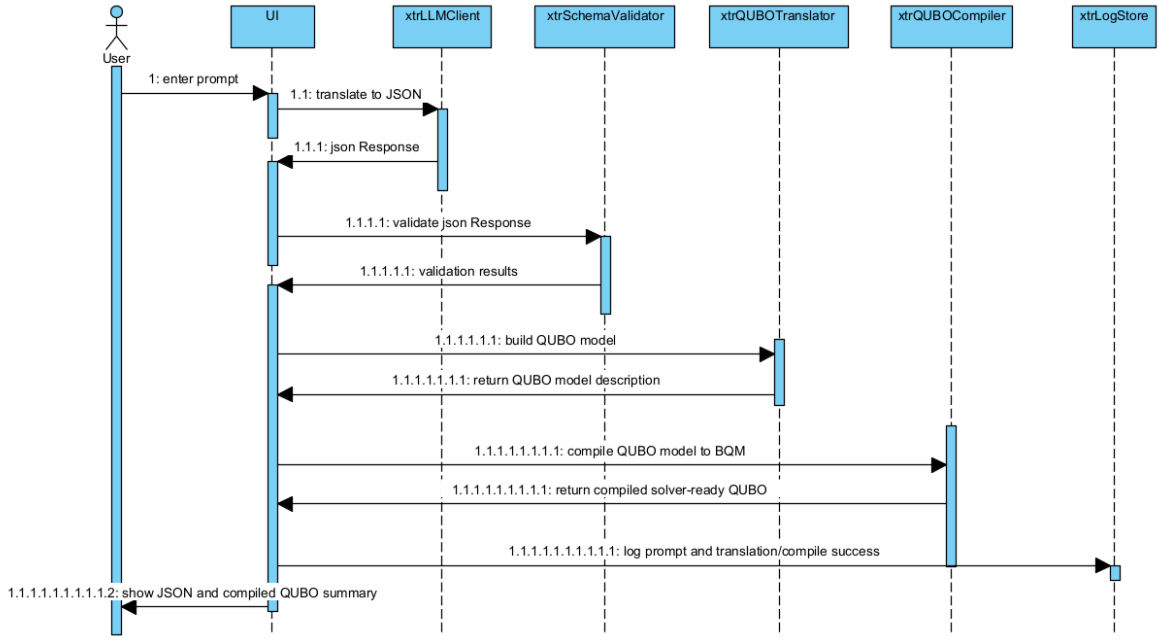

Figure 9.1: Sequence Diagram for ucTranslatePrompt (UC_1)

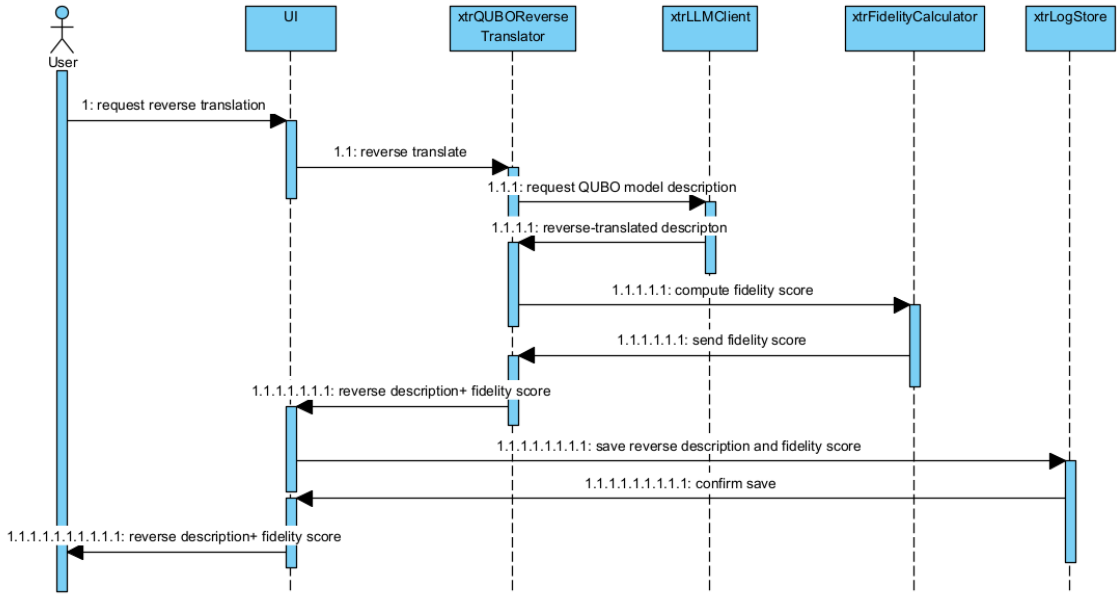

Figure 9.2: Sequence Diagram for ucReverseTranslate (UC_2)

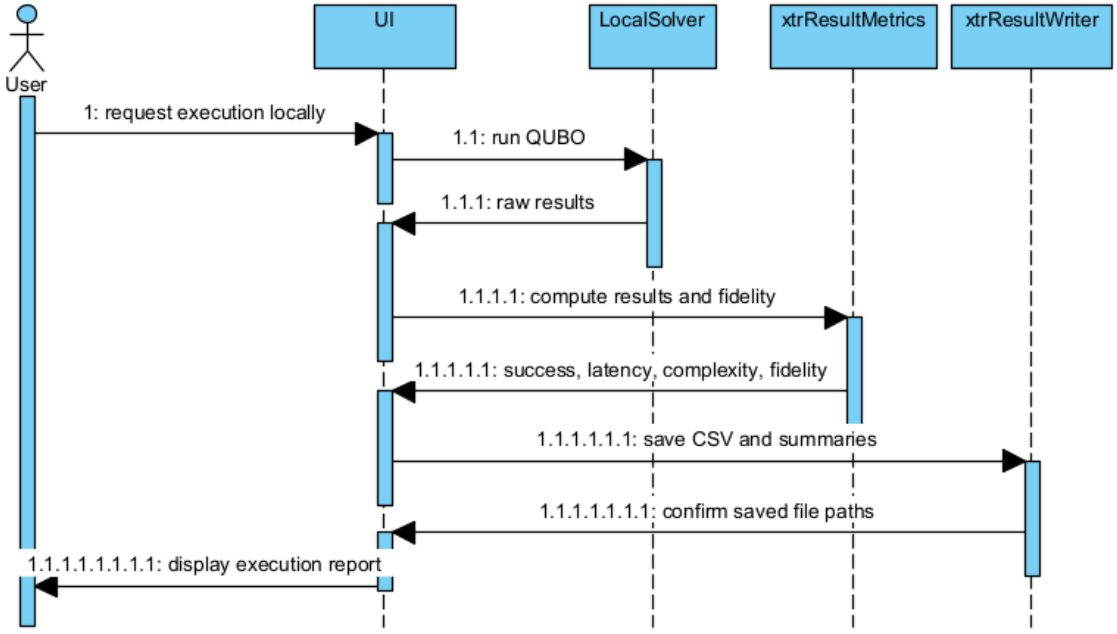

Figure 9.3: Sequence Diagram for ucExecuteReport (UC_3)

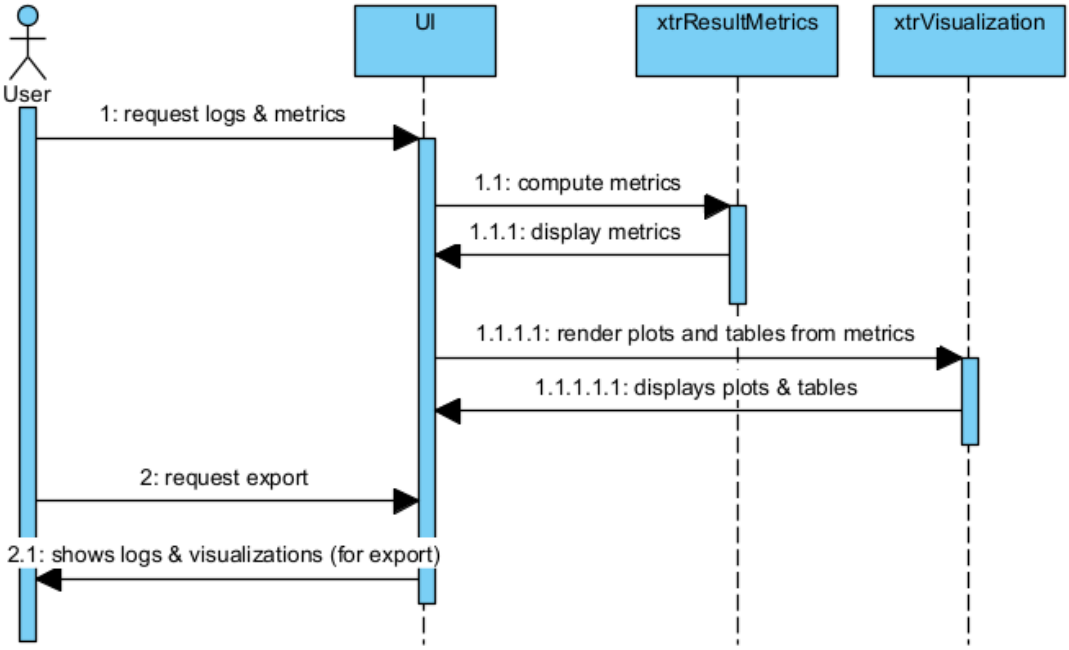

Figure 9.4: Sequence Diagram for ucViewLogsMetrics (UC_4)

Chapter 10

Implementation View

– Chan, Moks, Ponte

10.1 Class Diagram

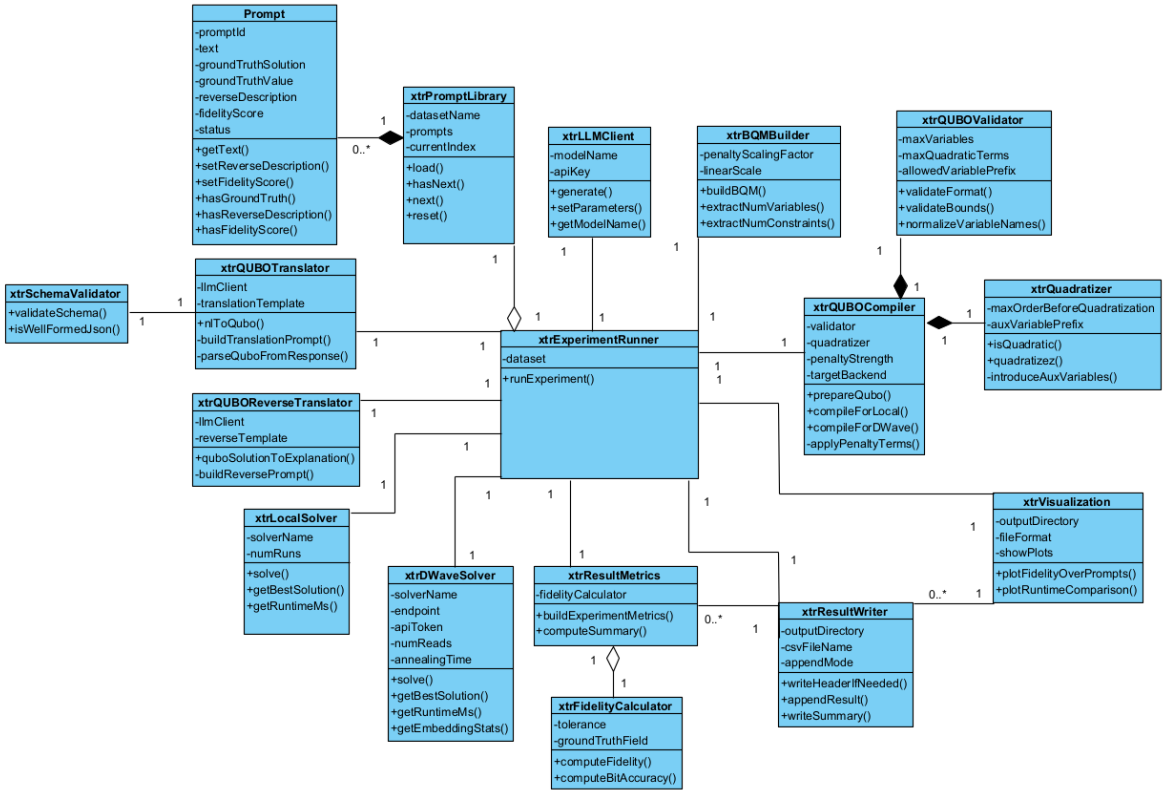

Figure 10.1: Class Diagram

Chapter 11

Development View

– Chan, Moks, Ponte

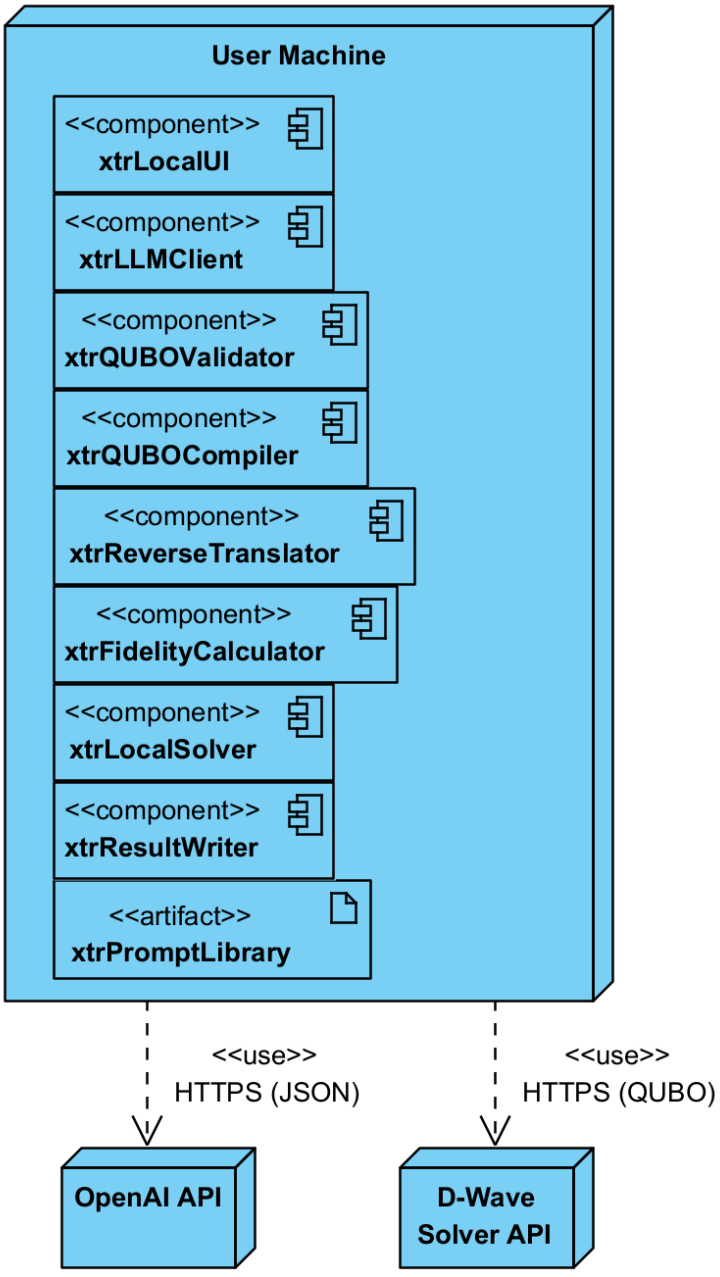

Figure 11.1: Deployment Diagram

Appendix A

Weekly Reports

– Chan, Moks, Ponte

A.1 Week Report 11 (11/14/2025)

Work Completed

- Finalized our decision to shift away from developing or evaluating an online/web-deployed interface and instead keep the system entirely local to the user's machine. We agreed that a locally hosted web interface is acceptable as an optional convenience layer, since it does not require external deployment.
- Updated the System Requirements and Constraints chapters to clarify that the project does not include public hosting, authentication, multi-user access, or large-scale usability studies. Any interface will run locally and remain single-user.
- Reviewed all existing use cases and user stories to ensure they still apply under a local-only model, making minor adjustments to remove assumptions about remote access or browser-based UX.
- Continued improving the core LLM-to-QUBO translation, validation, and reverse-translation pipeline so that it remains modular and stable regardless of the chosen local interface.
- Ensured the architecture description and diagrams are consistent with the local-first direction, where the interface is lightweight and the pipeline remains the primary focus of the system.

Planned Work for Next Week

- Complete all edits to the Requirements, Use Cases, and Design chapters so they clearly distinguish between the current local-only system and any potential future extension involving a cloud-hosted interface.
- Update or regenerate architecture figures, if needed, to illustrate the interaction model for a local UI or a locally hosted web interface running on `localhost`.
- Strengthen unit and integration tests around the pipeline components to ensure reliability during the final demo and report submission.
- Begin preparing the demonstration workflow (CLI sequence, notebook cells, or small local web interface walkthrough) for the final presentation.

Action Items

- Enoch: Review Requirements and Use Case chapters to remove outdated references to deployed web functionality and adjust traceability links accordingly.
- Benjamin: Update design text and any interface-related diagrams to emphasize the local interaction model and clarify how a local web interface fits into the architecture.
- Kyle: Update Weekly Reports and introductory sections to document the pivot clearly and ensure narrative consistency across earlier weeks and the final report.

Issues/Risks

- Some older text still references browser-based usability expectations, so careful editing is needed to avoid contradictions in the final draft.
- The shift away from a fully featured web interface reduces how much we can say about remote usability or interface testing.
- With limited time before the deadline, there is a risk of small mismatches between diagrams, requirements, and implementation if updates are not coordinated.

A.2 Week Report 10 (11/07/2025)

Work Completed

- Updated the project budget to include API key costs for OpenAI and D-Wave services.
- Met with a D-Wave advisor to discuss the Voyager quantum application and integration feasibility.
- Reviewed the fidelity scoring process to ensure consistent metrics across experiments.
- Made minor improvements to the project documentation and Use Case traceability.
- Configured a GitHub Actions autobuild to create versioned ZIP artifacts (e.g., QuantumOptimizationLLM_v1.0.1.zip) from the report PDF.
- Captured a screenshot of the successful autobuild run for documentation.

Planned Work for Next Week

- Begin preparing the next progress presentation.
- Continue working on validation consistency between translation and reverse translation modules.

Action Items

- Enoch: Finalize budget documentation and begin drafting presentation slides.
- Benjamin: Review and refine current code modules.
- Kyle: Continue testing QUBO outputs on the D-Wave hybrid solver and record performance metrics.

Issues/Risks

- API key expenses could increase with additional solver runs.
- Integration between local testing and D-Wave's Voyager environment may require additional configuration time.

Directory Packages Autobuild Screenshot

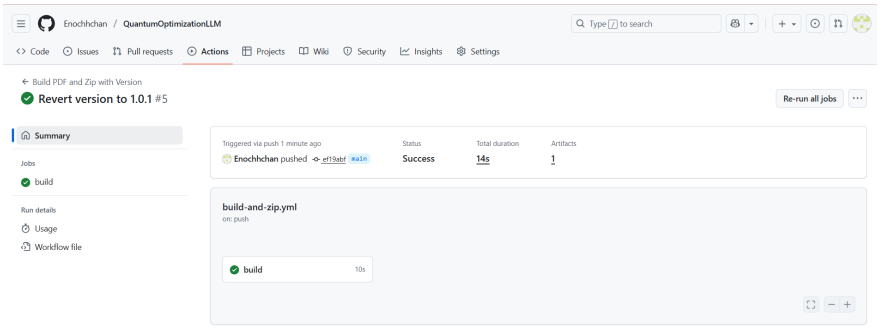

Figure A.1: GitHub Actions autobuild producing a versioned artifact `QuantumOptimizationLLM_v1.0.1.zip`.

A.3 Week Report 9 (10/31/2025)

Work Completed

- Met with D-Wave technical advisors at 3PM on 10/31/2025.
- Created Use Case Diagrams for each use case.
- Updated the requirements to include the fact that we are planning to deploy a web interface.

Planned Work for Next Week

- Introduce a new benchmark problem (Quadratic Assignment) per the request of D-Wave technical advisors.
- Experiment with the free capabilities of D-Wave hybrid solvers to validate the structure of our QUBO.
- Begin developing the web interface.
- Refine Use Case diagrams.

Action Items

- Enoch: Refine the Use Case diagrams.
- Benjamin: Determine how the web interface should be developed.
- Kyle: Incorporate the Quadratic Assignment problem into the rest of the experiments and begin testing.

Issues/Risks

- API costs of running additional experimentation.

A.4 Week Report 8 (10/24/2025)

Work Completed

- Finished Activity Diagrams for each of the use cases.
- Connected with a D-Wave representative (Jacob van Leenen) in regards to hybrid solver access.
- Corrections were made to the document based on feedback.

Planned Work for Next Week

- Meet with the D-Wave representative.
- Begin working on Use Case diagrams.
- Continue progress on the midterm presentation.

Action Items

- Enoch: Create Use Case diagrams.
- Benjamin: Organize the meeting with the D-Wave representative.
- Kyle: Work on the midterm presentation.

Issues/Risks

- Modular design may need tweaks.
- Midterm timeline pressure; coordination needed to keep UML and implementation in sync.

A.5 Week Report 7 (10/17/2025)

Work Completed

- Created Activity Diagrams for essential use cases and validated them against CRC cards.
- Refined object-oriented design; aligned class responsibilities with use case flows.
- Updated midterm deliverables (figures/tables).

Planned Work for Next Week

- Finalize Activity Diagrams and trace them to all use cases and requirements.
- Set up autobuild and versioning for the Hello World prototype (target: v0.1.#{changelist}).
- Continue searching for potential hybrid/quantum solvers.

Action Items

- Enoch: Finalize OO design; ensure UML and code skeletons align.
- Benjamin: Start module implementation; configure autobuild/versioning pipeline.

- Kyle: Compile Activity Diagrams into the report; draft Week 7 documentation updates.

Issues/Risks

- Limited access to hybrid hardware solver.
- Midterm timeline pressure; coordination needed to keep UML and implementation in sync.
- Ensuring traceability from CRC → Activity Diagrams → Use Cases → Requirements.

A.6 Week Report 6 (10/9/2025)

Work Completed

- Created CRC cards that demonstrate our initial OO design for the program.
- Formulated Use Cases and began going through them with our CRC cards to ensure the modular design is sufficient.
- Ran preliminary QUBO translation tests on D-Wave's exact solver to evaluate QUBO validity.

Planned Work for Next Week

- Begin working on Activity Diagrams for the essential use cases.
- Continue working on midterm deliverables.
- Make adjustments to OO design if necessary.

Action Items

- Enoch: Evaluate the modular design against use cases.
- Ben: Prepare midterm deliverables such as figures and tables.
- Kyle: Create the Activity Diagrams for essential use cases.

Issues/Risks

- Unsure of the suffiency of our current OO design.
- Still have concerns regarding how we will get access to D-Wave's quantum/hybrid solvers.

A.7 Week Report 5 (10/3/2025)

Work Completed

- Progressed on object-oriented refactoring by integrating modular translation, validation, and fidelity components to an extent.
- Expanded experiment dataset to include diverse prompt variations across all canonical problem types.
- Generated draft figures and tables summarizing schema validity, fidelity scores, and solver compatibility for midterm deliverables.

Planned Work for Next Week

- Incorporate reverse translation and semantic fidelity scoring into the experiment pipeline.
- Run preliminary QUBO translation tests on simulators to evaluate solver readiness.
- Polish midterm deliverables with finalized figures, tables, and updated methodology/results chapters.

Action Items

- Enoch: Implement reverse translation functions and fidelity scoring integration.
- Benjamin: Conduct initial QUBO translation and solver tests; refine experiment logging for semantic metrics.
- Kyle: Compile updated figures/tables into the report and draft midterm results discussion.

Issues/Risks

- Risk of incomplete semantic fidelity scoring before midterm deadline.
- Solver access and runtime limits may restrict depth of QUBO evaluation.
- Continued concern about API costs and token usage during extended testing.

A.8 Week Report 4 (09/25/2025)

Work Completed

- Implemented Team Assignment experiments and added prompt templates/validators.
- Began restructuring codebase with object-oriented design (class diagrams, UML sketches).
- Working to integrate preliminary object classes into experiment runner.

Planned Work for Next Week

- Finalize object-oriented refactor with fully modular translation and validation components.
- Expand testing dataset to larger prompt variations across all five canonical problem types.
- Prepare draft figures and tables for midterm deliverables.

Action Items

- Enoch: Complete UML diagrams and begin coding core classes.
- Benjamin: Refactor experiment runner to integrate modular classes and ensure compatibility.
- Kyle: Consolidate Team Assignment results, update report sections, and draft figures for midterm check-in.

Issues/Risks

- OOP refactor introduces risk of breaking existing scripts — testing and documentation required.
- API usage and token costs continue to be a concern for extended experiments.

A.9 Week Report 3 (09/19/2025)

Work Completed

- Conducted initial TSP and Graph Coloring experiments.
- Logged lower success rates and higher variability compared to Knapsack.
- Began drafting Results section outline.

Planned Work for Next Week

- Implement Team Assignment experiments.
- Restructure codebase with object-oriented design.

Action Items

- Enoch: Begin UML design for modular refactor.
- Benjamin: Continue expanding dataset and logs.
- Kyle: Extend experiments to Team Assignment.

Issues/Risks

- Lower success rates in Graph Coloring/TSP highlight model limitations.
- Risk of inconsistency across repeated runs.

A.10 Week Report 2 (09/12/2025)

Work Completed

- Reproduced Knapsack and Seating Assignment experiments from prior group's work.
- Hardened schema validator to detect missing objectives and malformed constraints.
- Implemented logging to CSV for prompt ID, status, latency, and fidelity score.
- Drafted methodology extension and pinned dependencies.

Planned Work for Next Week

- Extend experiments to TSP and Graph Coloring.
- Add timing metrics for translation, validation, and fidelity stages.
- Define clear success criteria for experiments.
- Generate figures for success rates, fidelity, and latency.

Action Items

- Enoch: Create TSP and Graph Coloring templates and validators.
- Benjamin: Add timers and generate plots.
- Kyle: Integrate results and update the report.

Issues/Risks

- API rate limits and costs may restrict experiments.
- Model variability affects schema validity.
- Dependency drift can cause errors.

A.11 Week Report 1 (09/05/2025)

Work Completed

- Reviewed the IEEE paper of the previous group, [1] and studied the LLM-to-QUBO translation pipeline.
- Examined schema validation, reverse translation, and fidelity scoring as described in the paper.
- Drafted the Introduction chapter with team member bios.
- Created the Weekly Reports appendix and updated the document history.

Planned Work for Next Week

- Reproduce a subset of experiments from the IEEE paper, starting with Knapsack and Seating Assignment.
- Configure the Python environment and experiment scripts.
- Begin drafting an extended methodology section.

Action Items

- Enoch: Set up Python environment and dependencies.
- Benjamin: Re-implement experiment logging scripts.
- Kyle: Draft updated methodology text and maintain version control.

Issues/Risks

- Difficulty replicating results without full access to prior environment.
- Latency and API limits could slow experimentation.

Glossary

BQM Binary Quadratic Model, solver-ready form of QUBO. 8

fidelity A measure of how well reverse-translated text preserves the original intent. 8

JSON A lightweight, human-readable text format for data interchange, used to transfer data between a server and a web application. 4

LLM Large Language Model, used for translating natural language into QUBO JSON. 3, 8

MustHave This defines the first highest priority requirement. All of the tasks, requirements, or anything that is marked this way are build in the current version. 8, 9, 10

Prompt Library A curated set of optimization prompts used for benchmarking and testing. 8

QUBO Quadratic Unconstrained Binary Optimization formulation used for optimization problems. 2, 4, 8

ShouldHave This defines the second highest priority requirement. The system should implement all of the tasks, requirements, or anything that is marked this way, but if resources are limited, it can be left out of the current version. Build in next version. 8, 10

Bibliography

[1] K. Sato, M. Singh, R. Dhadda, A. Rizzuto, A. Atrach, L. Xiao, and Y. Wang, "Trustworthy natural language interfaces for quantum optimization: A secure and resilient translation framework," in *IEEE Quantum Week (QCE)*. IEEE, 2025, pp. 1–12, available at: Download PDF.

Index